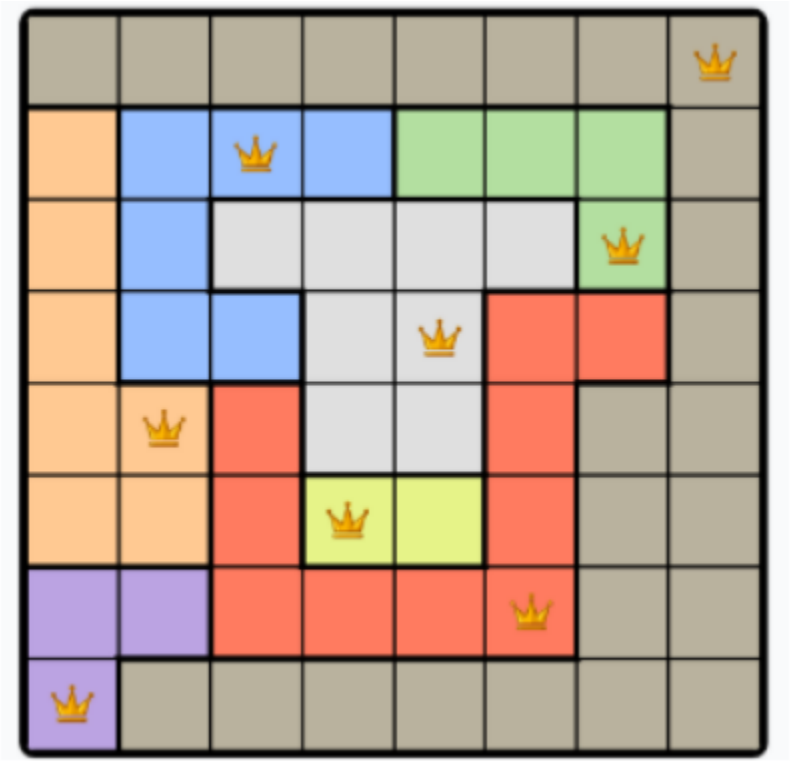
Laporan Tugas Kecil 1
IF 2211 Strategi Algoritma
Penyelesaian Permainan Queens Linkedin

Miguel Rangga Deardo Sinaga
13524069

1. Deskripsi Tugas

Queens adalah gim logika yang tersedia pada situs jejaring profesional LinkedIn. Tujuan dari gim ini adalah menempatkan queen pada sebuah papan persegi berwarna sehingga terdapat hanya satu queen pada tiap baris, kolom, dan daerah warna. Selain itu, satu queen tidak dapat ditempatkan bersebelahan dengan queen lainnya, termasuk secara diagonal. Tugas anda adalah membuat program yang dapat menemukan satu solusi penempatan queen pada suatu papan berwarna yang diberikan, atau menampilkan bahwa tidak ada solusi yang valid. Program melakukan pencarian solusi menggunakan algoritma brute force



2. Desain

Aplikasi ini dirancang menggunakan bahasa C++ untuk memaksimalkan performa komputasi pada algoritma brute force yang memerlukan iterasi dalam jumlah besar. Secara garis besar, arsitektur aplikasi terdiri dari komponen-komponen berikut:

1. Bahasa Pemrograman: C++
2. Compiler:g++ (GNU C++ Compiler)
3. Build System: Makefile
4. Library: Standard C++ Library (iostream, fstream, cstring, ctime)

2. 1 Struktur Program

Proyek disusun mengikuti struktur direktori standar:

```
TUCIL1_13524069/
├── src/                # Kode sumber C++
│   ├── main.cpp          # Program utama dan I/O
│   ├── QueensSolver.cpp   # Implementasi algoritma brute force
│   ├── QueensSolver.h     # Header file untuk solver
│   ├── BoardValidator.cpp # Validasi input papan
│   └── BoardValidator.h   # Header file untuk validator
├── build/              # Hasil kompilasi
│   └── queens_solver      # File executable
├── test/               # Test case input
│   ├── test1.txt
│   ├── test2.txt
│   ├── test3.txt
│   └── test_simple.txt
├── doc/                # Dokumentasi dan laporan
│   └── Laporan.pdf
├── Makefile            # Konfigurasi build
└── LICENSE
```

Program utama dibagi menjadi tiga komponen untuk memisahkan tanggung jawab:
- **main.cpp** berisi program utama untuk membaca input, menampilkan output, dan mengukur waktu eksekusi.
- **QueenSolver.cpp** berisi algoritma brute force untuk menempatkan queens dan validasi solusi.

- **BoardValidator.cpp** berisi fungsi-fungsi untuk memvalidasi input papan permainan.

2. 2 Cara kompilasi dan Menjalankan program

Untuk mengompilasi seluruh kode, jalankan perintah berikut pada root directory menggunakan WSL

```
make
```

Lalu, untuk menjalankan program setelah di kompilasi

```
make run
```

3. Implementasi Algoritma

3.1 Struktur Data

Board digunakan sebagai representasi fisik dari papan permainan. Struktur data ini diimplementasikan menggunakan array dua dimensi karakter. Setiap elemen dalam array menyimpan huruf region dari petak tersebut.

```
char board[MAX_ROWS][MAX_COLS + 1];       // Papan kerja
char originalBoard[MAX_ROWS][MAX_COLS + 1]; // Papan asli (backup)
char solution[MAX_ROWS][MAX_COLS + 1];     // Papan solusi
```

Position digunakan untuk menyimpan koordinat pada board. Position diimplementasikan menggunakan struct dengan “row” sebagai baris dan “col” sebagai kolom.

```
struct Position {
    int row;
    int col;
};
```

RegionCells digunakan untuk menyimpan daftar sel-sel yang termasuk dalam setiap region. Setiap region dapat memiliki maksimal “MAX_POSITIONS” sel.

```
Position regionCells[MAX_REGIONS][MAX_POSITIONS];
int regionCellCount[MAX_REGIONS];   // jumlah cells per region
char regionLetters[MAX_REGIONS];    // huruf region (A, B, C, ...)
int regionCount;                    // jumlah unique regions
```

choosenIdx digunakan untuk menyimpan index sel yang dipilih dari setiap region untuk penempatan queen saat ini.

```
int chosenIdx[MAX_REGIONS];
for(int r = 0; r < regionCount; r++) {
    chosenIdx[r] = 0;
}
```

3.2 Algoritma

Langkah 1 : Identifikasi region

```
// Identifikasi  region dan cells
void QueensSolver::getRegions() {
    regionCount = 0;
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            char c = originalBoard[i][j];
            if(c == '-' || c == '#') continue;
            // Cari apakah region sudah ada
            int regionIdx = -1;
            for(int r = 0; r < regionCount; r++) {
```

```
                if(regionLetters[r] == c) {
                    regionIdx = r;
                    break;
                }
            }
            // nambah region baru
            if(regionIdx == -1) {
                regionIdx = regionCount;
                regionLetters[regionCount] = c;
                regionCellCount[regionCount] = 0;
                regionCount++;
            }

            // nambah cell ke region
            int cellIdx = regionCellCount[regionIdx];
            regionCells[regionIdx][cellIdx].row = i;
            regionCells[regionIdx][cellIdx].col = j;
            regionCellCount[regionIdx]++;
        }
    }
}
```

Langkah 2 : Penempatan queen

Array chosenIdx[] menyimpan indeks cell yang dipilih dari setiap region untuk ditempatkan queen.

Contoh: chosenIdx = [1, 3, 0, 2]
chosenIdx[0] = 1 → cell ke-1 dari region A
chosenIdx[1] = 3 → cell ke-3 dari region B
chosenIdx[2] = 0 → cell ke-0 dari region C
chosenIdx[3] = 2 → cell ke-2 dari region D

Untuk setiap iterasi, queen ditempatkan dengan cara berikut:

```
for(int r = 0; r < regionCount; r++) {
    int cellIdx = chosenIdx[r];                    // Ambil indeks
cell yang dipilih
```

```
    int row = regionCells[r][cellIdx].row;          // Dapatkan
baris dari cell tersebut
    int col = regionCells[r][cellIdx].col;          // Dapatkan
kolom dari cell tersebut
    board[row][col] = '#';                          // Menaruh
Queen di posisi ini
}
```

Realisasi algoritma utama

Inisialisasi awal

```
int QueensSolver::solve() {
    iterationCount = 0;  // [1] Reset counter iterasi ke 0
```

```
    // Inisialisai array chosenIdx untuk cell yang dipilih tiap
region
    int chosenIdx[MAX_REGIONS];
    for(int r = 0; r < regionCount; r++) {
        chosenIdx[r] = 0;  // [3] Mulai dari cell pertama (index
0) di setiap region
    }
```

Penempatan Queen

```
int done = 0;
    while(!done) {
        copyBoard(board, originalBoard);  //
        // Place 1 queen per region berdasarkan chosenIdx
        for(int r = 0; r < regionCount; r++) {
            int cellIdx = chosenIdx[r];                    //Ambil
index cell yang dipilih
            int row = regionCells[r][cellIdx].row;
//Dapatkan baris dari cell
```

```
            int col = regionCells[r][cellIdx].col;
//Dapatkan kolom dari cell
            board[row][col] = '#';
//Tempatkan queen sementara
        }
```

Validasi queen sementara

```
iterationCount++;  // [10] Increment counter

        // Check validitas SETELAH placement lengkap (pure brute
force)
        if(isValidSolution(board)) {                    //Cek
apakah valid
            copyBoard(solution, board);                 //Simpan
solusi
            cout << "\n\n SOLUSI DITEMUKAN!" << endl;
            return 1;                                   //Return
sukses
        }
```

Fungsi atau Prosedur Pembantu

1. getRegion(), Prosedur ini melakukan scanning papan untuk mengidentifikasi semua region dan menyimpan koordinat sel-sel yang termasuk di dalamnya.

```
void QueensSolver::getRegions() {
    regionCount = 0;

    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            char c = originalBoard[i][j];
            if(c == '-' || c == '#') continue;

            // Cari apakah region sudah ada
            int regionIdx = -1;
            for(int r = 0; r < regionCount; r++) {
                if(regionLetters[r] == c) {
```

```
                    regionIdx = r;
                    break;
                }
            }

            // tambah region baru
            if(regionIdx == -1) {
                regionIdx = regionCount;
                regionLetters[regionCount] = c;
                regionCellCount[regionCount] = 0;
                regionCount++;
            }

            // Nambah cell
            int cellIdx = regionCellCount[regionIdx];
            regionCells[regionIdx][cellIdx].row = i;
            regionCells[regionIdx][cellIdx].col = j;
            regionCellCount[regionIdx]++;
        }
    }
}
```

2. isAttacking(int r1, int c1, int r2, int c2), fungsi ini mengambil koordinat dua queen dan mengembalikan `1` jika keduanya saling menyerang (baris sama, kolom sama, atau diagonal bersentuhan).

```
int QueensSolver::isAttacking(int r1, int c1, int r2, int c2)
{
    // Row sama
    if(r1 == r2) return 1;
    // Kolom sama
    if(c1 == c2) return 1;
    // Diagonal bersentuhan
    int rowDiff = r1 - r2;
    if(rowDiff < 0) rowDiff = -rowDiff;
    int colDiff = c1 - c2;
    if(colDiff < 0) colDiff = -colDiff;
    if(rowDiff == 1 && colDiff == 1) {
```

```
        return 1;
    }
    return 0;
}
```

3. isValidSolution(char currentBoard[MAX_ROWS][MAX_COLS + 1]), memeriksa apakah konfigurasi queen saat ini memenuhi semua aturan permainan.

```
int QueensSolver::isValidSolution(char
currentBoard[MAX_ROWS][MAX_COLS + 1]) {
    // Cari semua queens di board
    int queenPositions[MAX_POSITIONS][2];
    int queenCount = 0;

    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            if(currentBoard[i][j] == '#') {
                queenPositions[queenCount][0] = i;
                queenPositions[queenCount][1] = j;
                queenCount++;
            }
        }
    }

    // Jumlah queen harus sama dengan jumlah region
    if(queenCount != regionCount) return 0;

    // Check per row minimal 1 queen
    int queensPerRow[MAX_ROWS] = {0};
    for(int i = 0; i < queenCount; i++) {
        int row = queenPositions[i][0];
        queensPerRow[row]++;
        if(queensPerRow[row] > 1) return 0;
    }

    // Check per kolom 1 queen
    int queensPerCol[MAX_COLS] = {0};
    for(int i = 0; i < queenCount; i++) {
```

```
        int col = queenPositions[i][1];
        queensPerCol[col]++;
        if(queensPerCol[col] > 1) return 0;
    }

    // Check diagonal bersentuhan
    for(int i = 0; i < queenCount; i++) {
        for(int j = i + 1; j < queenCount; j++) {
            if(isAttacking(queenPositions[i][0],
queenPositions[i][1],
                           queenPositions[j][0],
queenPositions[j][1])) {
                return 0;
            }
        }
    }

    return 1;
}
```

4. Hasil Test Case

4.1 Test Case 1

Input :

```
AAABBCCCD
ABBBBCECD
ABBBDCECD
AAABDCCCD
BBBBDDDDD
FGGGDDHDD
FGIGDDHDD
FGIGDDHDD
FGGGDDHHH
```

Hasil :

```
mercury@LAPTOP-A9AOMELN:/mnt/c/Users/Miguel/OneDrive/Documents/TUCIL1_13524069$ make run
./build/queens_solver
───────QUEENS GAME SOLVER───────
Masukkan nama file (default: test/test1.txt): test/test1.txt
Membaca file: test/test1.txt

Parsing OK! Ukuran: 9x9

───────PAPAN AWAL (REGION)───────
A A A B B C C C D
A B B B B C E C D
A B B B D C E C D
A A A B D C C C D
B B B B D D D D D
F G G G D D H D D
F G I G D D H D D
F G I G D D H D D
F G G G D D H H H

Memulai pencarian solusi...

Jumlah region: 9
Region A: 8 cells
Region B: 14 cells
Region C: 10 cells
Region D: 25 cells
Region E: 2 cells
Region F: 4 cells
Region G: 10 cells
Region H: 6 cells
Region I: 2 cells

Total kombinasi yang akan dicek: 26880000

Mencari solusi...
[Progress 79%] Iterasi: 21410000 / 26880000

√ SOLUSI DITEMUKAN!
Iterasi akhir: 21415356 / 26880000

───────SOLUSI DITEMUKAN───────

── SOLUSI (# = QUEEN) ──
A A A B B C C # D
A B B B # C E C D
A B B B D C # C D
A # A B D C C C D
B B B B D # D D D
F G G # D D H D D
# G I G D D H D D
F G # G D D H D D
F G G G D D H H #

Waktu pencarian: 2260 ms
Banyak kasus yang ditinjau: 21415356 kasus

Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
```

4.2 Test Case 2

Input :

```
AAAABBBB
AACCCCBB
AACCDDBB
AACCDDBB
EEFFDDGG
EEFFHHGG
EEFFHHGG
EEFFHHGG
```

Hasil :

```
./build/queens_solver
────────QUEENS GAME SOLVER────────
Masukkan nama file (default: test/test1.txt): test/test2.txt
Membaca file: test/test2.txt

Parsing OK! Ukuran: 8x8

────────PAPAN AWAL (REGION)────────
A A A A B B B B
A A C C C C B B
A A C C D D B B
A A C C D D B B
E E F F D D G G
E E F F H H G G
E E F F H H G G
E E F F H H G G

Memulai pencarian solusi...

Jumlah region: 8
Region A: 10 cells
Region B: 10 cells
Region C: 8 cells
Region D: 6 cells
Region E: 8 cells
Region F: 8 cells
Region G: 8 cells
Region H: 6 cells

Total kombinasi yang akan dicek: 14745600

Mencari solusi...
[Progress 4%] Iterasi: 670000 / 14745600

√ SOLUSI DITEMUKAN!
Iterasi akhir: 670260 / 14745600

────────SOLUSI DITEMUKAN────────

── SOLUSI (# = QUEEN) ──
# A A A B B B B
A A C C C C # B
A A # C D D B B
A A C C # D B B
E # F F D D G G
E E F # H H G G
E E F F H H G #
E E F F H # G G

Waktu pencarian: 58 ms
Banyak kasus yang ditinjau: 670260 kasus

Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
```

4.3 Test Case 3

Input :

```
AABBBBBB
AABBBCCC
AABBCCCC
ADDECCCC
ADDEEEEE
FFDIIIII
FFIIIIJJ
FFIIJJJJ
```

Hasil :

```
mercury@LAPTOP-A9AOMELN:/mnt/c/Users/Miguel/OneDrive/Documents/TUCIL1_13524069$ make run
./build/queens_solver
───────QUEENS GAME SOLVER───────
Masukkan nama file (default: test/test1.txt): test/test3.txt
Membaca file: test/test3.txt

Parsing OK! Ukuran: 8x8

───────PAPAN AWAL (REGION)───────
A A B B B B B B
A A B B B C C C
A A B B C C C C
A D D E C C C C
A D D E E E E E
F F D I I I I I
F F I I I I J J
F F I I J J J J

Memulai pencarian solusi...

Jumlah region: 8
Region A: 8 cells
Region B: 11 cells
Region C: 11 cells
Region D: 5 cells
Region E: 6 cells
Region F: 6 cells
Region I: 11 cells
Region J: 6 cells

Total kombinasi yang akan dicek: 11499840

Mencari solusi...
[Progress 8%] Iterasi: 960000 / 11499840

√ SOLUSI DITEMUKAN!
Iterasi akhir: 965670 / 11499840

───────SOLUSI DITEMUKAN───────

── SOLUSI (# = QUEEN) ──
# A B B B B B B
A A B # B C C C
A A B B C # C C
A D # E C C C C
A D D E # E E E
F F D I I I # I
F # I I I I J J
F F I I J J J #

Waktu pencarian: 79 ms
Banyak kasus yang ditinjau: 965670 kasus

Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
```

4.4 Test Case 4

Input :

```
AABBCC
AABBCC
DDEEFF
DDEEFF
GGHHII
GGHHII
```

Hasil :

```
./build/queens_solver
———QUEENS GAME SOLVER———
Masukkan nama file (default: test/test1.txt): test/test4.txt
Membaca file: test/test4.txt

Parsing OK! Ukuran: 6x6

———PAPAN AWAL (REGION)———
A A B B C C
A A B B C C
D D E E F F
D D E E F F
G G H H I I
G G H H I I

Memulai pencarian solusi...

Jumlah region: 9
Region A: 4 cells
Region B: 4 cells
Region C: 4 cells
Region D: 4 cells
Region E: 4 cells
Region F: 4 cells
Region G: 4 cells
Region H: 4 cells
Region I: 4 cells

Total kombinasi yang akan dicek: 262144

Mencari solusi...
[Progress 99%] Iterasi: 260000 / 262144

✗ TIDAK ADA SOLUSI
Iterasi akhir: 262144 / 262144

=———TIDAK ADA SOLUSI———
Waktu pencarian: 16 ms
Banyak kasus yang ditinjau: 262144 kasus
mercury@LAPTOP-A9AOMELN:/mnt/c/Users/Miguel/OneDrive/Documents/TUCIL1_13524069$
```

4.5 Test Case 5

input:

```
AABBCCDD
AEEBFCDG
HEIIIFDG
HEJJFFGG
HHJKKLGM
NJKKLLMM
NNOOLMMM
NPPPOOOP
```

Hasil :

```
./build/queens_solver
———QUEENS GAME SOLVER———
Masukkan nama file (default: test/test1.txt): test/test6.txt
Membaca file: test/test6.txt

Parsing OK! Ukuran: 8x8

———PAPAN AWAL (REGION)———
A A B B C C D D
A E E B F C D G
H E I I I F D G
H E J J F F G G
H H J K K L G M
N J K K L L M M
N N O O L M M M
N P P P O O O P

Memulai pencarian solusi...

Jumlah region: 16
Region A: 3 cells
Region B: 3 cells
Region C: 3 cells
Region D: 4 cells
Region E: 4 cells
Region F: 4 cells
Region G: 5 cells
Region H: 4 cells
Region I: 3 cells
Region J: 4 cells
Region K: 4 cells
Region L: 4 cells
Region M: 6 cells
Region N: 4 cells
Region O: 5 cells
Region P: 4 cells

Total kombinasi yang akan dicek: 3185049600

Mencari solusi...
[Progress 99%] Iterasi: 3185040000 / 3185049600    Iterasi akhir: 3185049600 / 3185049600

=———TIDAK ADA SOLUSI———
Waktu pencarian: 294217 ms
Banyak kasus yang ditinjau: 3185049600 kasus
```

5. Link github

https://github.com/miguelsinaga/TUCIL1_13524069.git